Foundations of Spatial AI

Lecture 08 — Neural Radiance Fields

Detailed lecture notes

Based on the slides by Emanuele Giacomini, Sapienza University of Rome (October 18, 2026)

Abstract

These notes follow the flow of the lecture slides: we recall *computational volume rendering* (ray marching), then ask how a volume can be *represented* so that density and emitted light can be queried at any 3D point — a voxel grid, an MLP (the Neural Radiance Field), voxels with spherical-harmonic colours (Plenoxels) and finally feature hash-grids with a tiny MLP (Instant-NGP). Along the way we discuss positional encoding, hierarchical and inverse-depth sampling, view-dependent effects and anti-aliasing (mip-NeRF). Slide ranges are given in each section title. No companion book chapter was provided, so the formal statements (definitions, lemmas, proofs) were derived from the slides and from the cited papers (NeRF, Fourier Features, mip-NeRF, Plenoxels, Instant-NGP).

Contents

1 Neural Radiance Fields *(slides 1–2)* **3**

2 Computational Volume Rendering: Ray Marching *(slides 3–8)* **4**

2.1 Ray marching: sum over segments *(slides 3–6)* 4

2.2 Transmittance as a recursion *(slides 4–6)* 5

2.3 Enabling background radiance *(slides 7)* 6

2.4 Summary: computational volume rendering *(slides 8)* 6

3 Volumes: Rendering and Representation *(slides 9–10)* **7**

3.1 Option 1: a grid — Grid Representation *(slides 11–17)* 8

4 Neural Radiance Fields *(slides 18–20)* **10**

5 Learning Neural Radiance Fields *(slides 21–24)* **11**

5.1 From pixels to rays *(slides 21–22)* 11

5.2 Mini-batches of rays *(slides 23)* 12

6 Learning Neural Radiance Fields: An Initial Attempt *(slides 25)* **12**

7 Modeling High Frequencies: A Toy Problem *(slides 26–30)* **13**

8 Uniform to Hierarchical Sampling *(slides 31–33)* **15**

9 Depth vs Inverse-Depth Sampling *(slides 34–36)* **17**

9.1 Normalised device coordinates *(slides 36)* 17

10 Novel-View Synthesis and View-Dependent Effects *(slides 37–41)* **18**

10.1 Novel-View Synthesis *(slides 37)* 18

10.2 View-Dependent Effects *(slides 38–40)* 18

10.3 Learning Neural Radiance Fields: a simple recipe *(slides 41)* 20

11 Is ray sampling accurate? *(slides 42)* **20**

12 Pixels as Cones of Rays *(slides 43–46)* **21**

13 Volumes: Rendering and Representation — Option 2: An MLP, revisited *(slides 47)* **23**

14 Option 3: Voxels with view-dependent radiance — Do We Need Neural Radiance? *(slides 48–51)* **23**

14.1 Spherical harmonics for view-dependent colour *(slides 49)* 23

14.2 The Plenoxels pipeline *(slides 50)* 24

14.3 NeRF vs. Plenoxels *(slides 51)* 25

15 Towards Hybrid Representations *(slides 52–54)* **25**

16 Summary *(slides 55)* **27**

1 Neural Radiance Fields *(slides 1–2)*

The goal of the lecture is summarised in one line on slide 2:

$$\text{posed images} \longrightarrow \text{a neural network} \longrightarrow \text{novel views.}$$

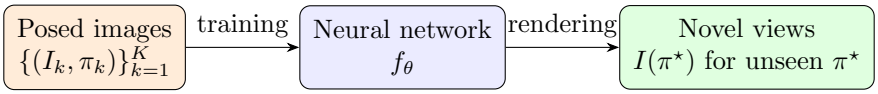


Figure 1: The NeRF problem at a glance (redrawn from slide 2): learn f_θ from images with known cameras π_k, then render images from cameras never seen during training.

Insert image here — take it from slide 2

Figure 2: Left: nine of the posed training images of the NeRF "lego" scene; right: a novel view rendered by `nerfacto` (Nerfstudio). Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

Definition 1.1 (Novel-view synthesis)**.** Given images $I_1, \dots, I_K$ of a static scene together with their camera parameters $\pi_1, \dots, \pi_K$ (intrinsics and pose), *novel-view synthesis* (NVS) is the task of producing the image $I(\pi^\star)$ that a camera with parameters $\pi^\star \notin \{\pi_k\}$ would see.

Definition 1.2 (Radiance field)**.** A *radiance field* is a function $(\boldsymbol{x}, \boldsymbol{\omega}) \mapsto (\sigma, \boldsymbol{c})$ that assigns to every 3D point $\boldsymbol{x} \in \mathbb{R}^3$ and viewing direction $\boldsymbol{\omega} \in \mathbb{S}^2$ a volume density $\sigma \in \mathbb{R}^+$ and an emitted colour (radiance) $\boldsymbol{c} \in [0,1]^3$. A *Neural* Radiance Field (NeRF) is a radiance field parameterised by a neural network f_θ.

Intuition

Think of the scene as a cloud of coloured, glowing smoke. $\sigma(\boldsymbol{x})$ says "how thick is the smoke here?" and $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega})$ says "what colour does it glow when seen from direction $\boldsymbol{\omega}$?". A photograph is what you get by looking at this cloud through a pinhole. Training means adjusting the cloud until all the photographs we have are reproduced; because the cloud is a genuine 3D object, we can then look at it from anywhere.

Roadmap of the lecture. The method has two ingredients that the slides keep separate: (i) a *rendering algorithm* that turns density and emission along a ray into a pixel colour (Sections 2–3), and (ii) a *representation* of the volume from which density and emission can be queried pointwise (grid, MLP, voxels + spherical harmonics, hash-grid; Sections 3–13).

2 Computational Volume Rendering: Ray Marching *(slides 3–8)*

2.1 Ray marching: sum over segments *(slides 3–6)*

A camera at $\boldsymbol{x}$ looks in direction $\boldsymbol{\omega}$. We discretise the ray $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{\omega}$ into N segments delimited by $N+1$ sample points $t_1 < t_2 < \cdots < t_{N+1}$ (slide 5 draws them uniformly; slide 6 shows that they need not be). We write

$$\boldsymbol{x}_{t_i} = \boldsymbol{x} + t_i\boldsymbol{\omega}, \qquad \Delta t_i \equiv t_{i+1} - t_i, \qquad \sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}).$$

Slide 3 states the target: the radiance reaching the camera is a sum of one *contribution per segment*.

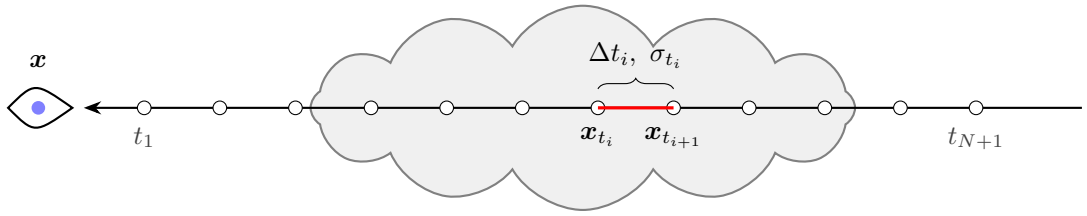


Figure 3: Ray marching through an emitting/absorbing medium (redrawn from slides 3–6; original ray-marching figures by I. Gkioulekas). Within each segment the density σ_{t_i} and the emission $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$ are assumed constant.

Medium model. The medium is *absorbing and emitting*: every infinitesimal piece of volume emits light (colour) and absorbs part of the light passing through it. Absorption is governed by the density σ and the *transmittance*.

Definition 2.1 (Transmittance)**.** For two points $\boldsymbol{x}_a = \boldsymbol{x}_{t_a}$ and $\boldsymbol{x}_b = \boldsymbol{x}_{t_b}$ on a ray, with $t_a < t_b$, the transmittance is the fraction of light that survives the trip between them,

$$T(\boldsymbol{x}_a, \boldsymbol{x}_b) = \exp\Big(-\int_{t_a}^{t_b} \sigma(\boldsymbol{x}_s)\,\mathrm{d}s\Big) \in (0, 1].$$

Lemma 2.1 (A homogeneous emitting segment)**.** *Consider a segment of length Δt with constant density σ and constant emission L_e. The radiance it delivers to its near end, ignoring everything outside the segment, is*

$$\int_0^{\Delta t} e^{-\sigma s}\,\sigma\,L_e\,\mathrm{d}s = \big(1 - e^{-\sigma\Delta t}\big)L_e.$$

Proof. Light emitted at depth s inside the segment is attenuated by $e^{-\sigma s}$ before leaving it, and the amount emitted per unit length is σL_e. Hence $\int_0^{\Delta t} \sigma L_e e^{-\sigma s}\,\mathrm{d}s = L_e\,[-e^{-\sigma s}]_0^{\Delta t} = L_e(1-e^{-\sigma\Delta t})$. □

Intuition

The factor $\alpha = 1 - e^{-\sigma\Delta t}$ is the segment's *opacity*: the probability that the ray is "stopped" (and thus sees the emitted colour) inside it. Thin or dilute segments ($\sigma\Delta t \ll 1$) have $\alpha \approx \sigma\Delta t$; a thick one ($\sigma\Delta t \gg 1$) has $\alpha \to 1$ and is a solid wall. Example: $\sigma\Delta t = 0.5 \Rightarrow \alpha = 0.39$, and $\sigma\Delta t = 3 \Rightarrow \alpha = 0.95$.

2.2 Transmittance as a recursion *(slides 4–6)*

The light emitted by segment i must still travel through all the segments between it and the camera; this is where $T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ enters (slide 4).

Lemma 2.2 (Multiplicativity of transmittance)**.** *With $T_i \equiv T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ we have the base case $T_1 = 1$ and the recursion*

$$T_i = T_{i-1}\, e^{-\sigma_{t_{i-1}} \Delta t_{i-1}} \quad (i \geq 2), \qquad \textit{hence} \qquad T_i = \exp\Big(- \sum_{j<i} \sigma_{t_j} \Delta t_j \Big).$$

(Slide 4 writes Δt; slide 6 shows that it may depend on i.)

Proof. $T_1 = 1$ because no medium lies between the camera and the first sample. For $i \geq 2$, split the integral in T at t_{i-1}: $\int_{t_1}^{t_i} \sigma = \int_{t_1}^{t_{i-1}} \sigma + \int_{t_{i-1}}^{t_i} \sigma$, and the exponential of a sum is a product. On the last segment σ is constant, so the second integral equals $\sigma_{t_{i-1}} \Delta t_{i-1}$. Unrolling the recursion gives the closed form. □

Theorem 2.1 (Discrete volume rendering)**.** *For a ray with segments as above, the radiance reaching the camera is*

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) \left(1 - e^{-\sigma_{t_i} \Delta t_i}\right) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}) = \sum_{i=1}^{N} w_i\, \boldsymbol{c}_i,$$

where $w_i = T_i \alpha_i$, $\alpha_i = 1 - e^{-\sigma_{t_i} \Delta t_i}$ and $\boldsymbol{c}_i = L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$.

Proof. By Lemma 2.1 segment i emits $\alpha_i \boldsymbol{c}_i$ at its own near end; by Lemma 2.2 a fraction T_i of it survives the way to the camera. Light emission is additive, so we sum over segments. □

Intuition

Standing at the camera you walk along the ray. At every segment you ask two questions: "did the light from here survive everything in front of me?" (T_i) and "does this segment actually emit anything?" ($\alpha_i \boldsymbol{c}_i$). The weight w_i is "how much segment i contributes to the pixel"; this is the quantity that colours the segments red/yellow on slides 32–33 and that will drive hierarchical sampling (Section 7).

Algorithm 2.1 (Ray marching, slide 5)**.**

1. Draw samples along the ray (N segments, i.e. $N + 1$ points); slide 5 uses uniform spacing.
2. Compute the transmittance T_i between the camera and each sample (recursion of Lemma 2.2).
3. Aggregate the contributions $T_i \alpha_i \boldsymbol{c}_i$ over all segments to obtain the overall radiance (the pixel colour).

Example 2.1 (Worked numbers)**.** Take $N = 4$ segments of length $\Delta t = 0.5$, densities $\sigma = (0.2, 2, 3, 0.4)$ and grey-level emissions $L_e = (0.2, 0.9, 0.5, 0.3)$. Then $\alpha = (0.095, 0.632, 0.777, 0.181)$, $T = (1, 0.905, 0.333, 0.074)$ and so $w = T\alpha = (0.095, 0.572, 0.259, 0.014)$. The pixel value is $0.095 \cdot 0.2 + 0.572 \cdot 0.9 + 0.259 \cdot 0.5 + 0.014 \cdot 0.3 \approx 0.667$. Segment 2 dominates: it is dense *and* close to the camera. Segment 4 is hidden: T_4 is only 0.074 even though $\alpha_4 = 0.18$.

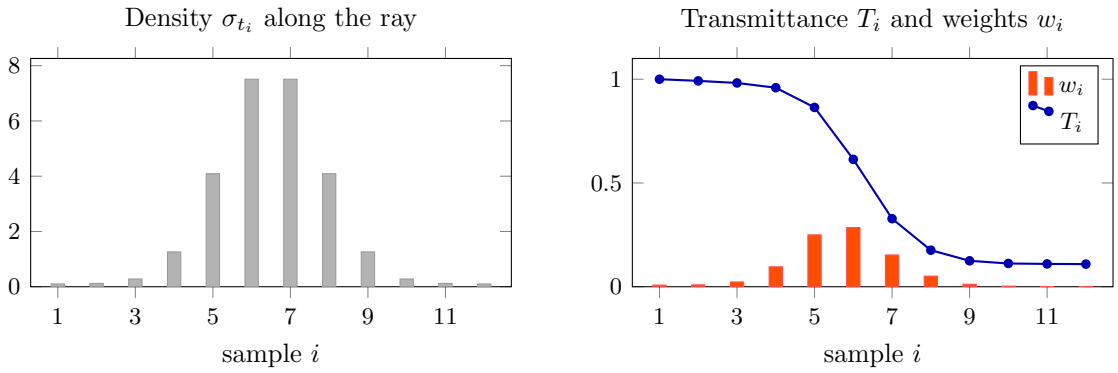


Figure 4: Original illustration (not in the slides): a ray with 12 segments of length $1/12$ crossing a dense blob centred at the middle of the ray ($\sigma_{t_i} = 0.1 + 8e^{-((t_i-0.5)/0.15)^2}$ at segment midpoints). Transmittance drops while the ray crosses the blob and the weights w_i peak at the near side of the blob — the first dense samples hide the ones behind. $\sum_i w_i = 0.89$ and $T_{N+1} = 0.11$.

2.3 Enabling background radiance *(slides 7)*

If the ray does not end inside the medium but hits an opaque surface or a background (the grey wall $\boldsymbol{x}_z$ on slide 7), the light that survives the whole medium adds one more term:

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i})\big(1 - e^{-\sigma_{t_i}\Delta t}\big) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}) \;+\; T(\boldsymbol{x}, \boldsymbol{x}_{t_{N+1}})\, L(\boldsymbol{x}_z, \boldsymbol{\omega}).$$

Proposition 2.1 (Weights sum to one)**.** *With $w_i = T_i\alpha_i$ we have $w_i = T_i - T_{i+1}$ and therefore*

$$\sum_{i=1}^{N} w_i + T_{N+1} = 1.$$

Consequently the rendered colour is a convex combination *of the segment colours and the background colour, and $T_{N+1} = 1 - \sum_i w_i$ is the* background mask*.*

Proof. By Lemma 2.2, $T_{i+1} = T_i e^{-\sigma_{t_i}\Delta t_i} = T_i(1 - \alpha_i)$, so $T_i - T_{i+1} = T_i\alpha_i = w_i$. The sum telescopes: $\sum_{i=1}^{N} w_i = T_1 - T_{N+1} = 1 - T_{N+1}$. □

Intuition

This is *alpha compositing*. Every segment "paints" with opacity α_i over what is behind it; whatever the medium lets through (T_{N+1}) shows the background. Numerically, in Example 2.1 the weights sum to 0.939 and $T_5 = 0.061$; with a white background ($L = 1$) the pixel becomes $0.667 + 0.061 \cdot 1 = 0.728$. For synthetic scenes with a white background one therefore renders $\hat{C} = \sum_i w_i \boldsymbol{c}_i + (1 - \sum_i w_i) \cdot 1$. In the same way, T_{N+1} is a soft "this ray hits nothing" mask.

2.4 Summary: computational volume rendering *(slides 8)*

If we can compute (a) a per-point density $\sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i})$ and (b) a per-point, per-direction emitted light $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$, then we can render *any* ray through the medium, hence an image from any camera viewpoint using $H \times W$ rays (one per pixel).

Proposition 2.2 (Differentiability of the renderer)**.** *Let* $\hat{C} = \sum_i w_i \boldsymbol{c}_i + T_{N+1} L_{\text{bg}}$. *Then*

$$\frac{\partial \hat{C}}{\partial \boldsymbol{c}_i} = w_i, \qquad \frac{\partial \hat{C}}{\partial \sigma_{t_k}} = \Delta t_k \Big(T_{k+1}\, \boldsymbol{c}_k - \sum_{i>k} w_i \boldsymbol{c}_i - T_{N+1} L_{\text{bg}} \Big).$$

Proof. The first identity is immediate. For σ_{t_k}: α_k depends on it with $\partial \alpha_k / \partial \sigma_{t_k} = \Delta t_k e^{-\sigma_{t_k} \Delta t_k}$, so the term $w_k \boldsymbol{c}_k$ contributes $T_k \Delta t_k e^{-\sigma_{t_k} \Delta t_k} \boldsymbol{c}_k = \Delta t_k T_{k+1} \boldsymbol{c}_k$. Moreover T_i for every $i > k$ (and T_{N+1}) contains $e^{-\sigma_{t_k} \Delta t_k}$, so $\partial T_i / \partial \sigma_{t_k} = -\Delta t_k T_i$ and $w_i = T_i \alpha_i$ gives $-\Delta t_k w_i \boldsymbol{c}_i$; likewise for the background term. □

Intuition

Increasing the density at sample k has two opposite effects (slide 8's red note: the pipeline is differentiable). It makes sample k's own colour more visible (positive term) but it also hides everything behind it (negative terms). Gradient descent on a photometric loss thus "pushes density" to where it explains the pixel and removes it where it would wrongly occlude. The same holds for the camera parameters whenever density and emission are differentiable functions of position and direction.

3 Volumes: Rendering and Representation *(slides 9–10)*

Slides 9–10 split the problem into two boxes:

$$\underbrace{\begin{matrix} \sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}) \\ L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}) \end{matrix}}_{\text{representation: "How to represent volumes?"}} \longrightarrow \underbrace{L(\boldsymbol{x}, \boldsymbol{\omega})}_{\text{rendering algorithm (done)}}$$

The rendering algorithm is fixed; what remains is a *representation* from which the density and the emitted light can be computed pointwise.

Insert image here — take it from slide 9

Figure 5: A volumetric example (smoke-like plume): volumes can be rich and view-dependent, and we need a representation that can encode them. Figure credits as on the slide.

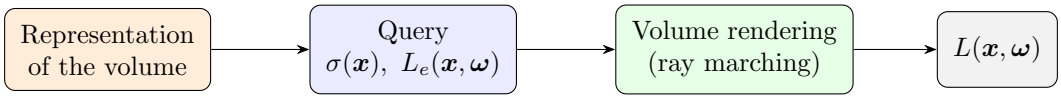


Figure 6: Division of labour in slides 9–10: the representation only has to answer pointwise queries; ray marching does the rest.

3.1 Option 1: a grid — Grid Representation *(slides 11–17)*

Definition 3.1 (Voxel-grid volume)**.** A grid stores a positive scalar in each voxel, $\sigma[x, y, z] \in \mathbb{R}^+$ (the density), and — for emission — a colour $\boldsymbol{c}[x, y, z] \in [0, 1]^3$. Since the lookup of an arbitrary real-valued point $\boldsymbol{x}$ is not a voxel index, slide 11 asks: *how to obtain $\sigma(\boldsymbol{x})$ for arbitrary $\boldsymbol{x}$?* The answer is interpolation.

Trilinear interpolation *(slides 12)*

Let $\boldsymbol{x}$ fall into a cell whose eight corners carry the values σ_{ijk}, $i, j, k \in \{0, 1\}$. Let $(u, v, w) \in [0, 1]^3$ be the local coordinates of $\boldsymbol{x}$ in the cell, and set $u_1 = u,\ u_0 = 1 - u$ (same for v, w).

Definition 3.2 (Trilinear interpolation)**.** $\sigma(\boldsymbol{x}) = \sum_{i,j,k \in \{0,1\}} u_i\, v_j\, w_k\, \sigma_{ijk}$. It is the 3D equivalent of bilinear interpolation.

Lemma 3.1 (Properties of trilinear interpolation)**.** *(i) The weights $\lambda_{ijk} = u_i v_j w_k$ are non-negative and sum to* 1*. (ii) The weight of a corner equals the volume of the box spanned by $\boldsymbol{x}$ and the* opposite *corner. (iii) σ is linear in each of u, v, w separately, reproduces the corner values, and is continuous across cell boundaries. (iv) $\partial\sigma(\boldsymbol{x})/\partial\sigma_{ijk} = \lambda_{ijk}$: the interpolation is differentiable w.r.t. the stored corner values.*

Proof. (i) $\sum \lambda_{ijk} = (u_0 + u_1)(v_0 + v_1)(w_0 + w_1) = 1$. (ii) The box between $\boldsymbol{x}$ and the corner $(0, 0, 0)$ has side lengths u, v, w, so its volume is $uvw = \lambda_{111}$; the other corners follow by symmetry (e.g. $\lambda_{000} = (1 - u)(1 - v)(1 - w)$ is the box towards $(1, 1, 1)$). (iii) Fix v, w: σ is a convex combination $(1 - u)a + ub$, linear in u; at a corner one λ equals 1 and the others vanish. On a shared face, say $u = 1$, the formula only involves the four corners of that face, so neighbouring cells agree. (iv) Linearity in the σ_{ijk}. □

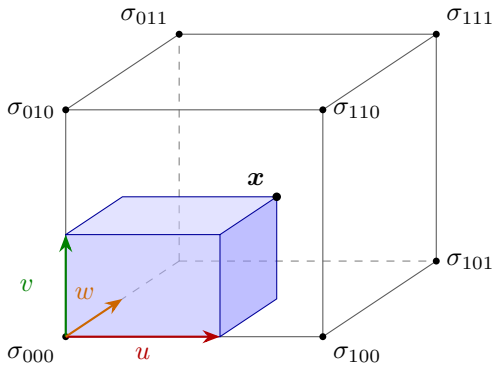


Figure 7: Trilinear interpolation (redrawn from slide 12). The shaded box between $\boldsymbol{x}$ and corner $(0, 0, 0)$ has volume uvw and is the weight of the *opposite* corner σ_{111}; the other seven weights are the volumes of the other seven sub-boxes, which tile the cell.

Intuition

Imagine standing in the cell and drawing the three planes through yourself parallel to the faces: they cut the cell into eight little boxes. Each corner is weighted by the box *on the other side* of you — the closer you are to a corner, the bigger the opposite box, hence the larger its weight. In 1D this is just $(1 - u)a + ub$; e.g. for $u = 0.25$ the nearer value gets weight

0.75. In 3D with $(u, v, w) = (0.6, 0.45, 0.5)$ as in the figure, $\lambda_{111} = 0.6 \cdot 0.45 \cdot 0.5 = 0.135$ and $\lambda_{000} = 0.4 \cdot 0.55 \cdot 0.5 = 0.11$.

Emission on the grid *(slides 13–15)*

Q: how do we model view-dependent emission $L_e(\boldsymbol{x}, \boldsymbol{\omega})$? The grid answer (slide 13) is to *ignore* view dependence: $L_e(\boldsymbol{x}, \boldsymbol{\omega}) \equiv \boldsymbol{c}(\boldsymbol{x})$. Slide 14 therefore stores a colour $\boldsymbol{c}[x, y, z] \in [0, 1]^3$ next to the density in every voxel, and trilinear interpolation is applied to both.



Figure 8: Left: a chair stored as a coloured voxel grid (slide 14). Right: an emission-only rendering of a cloud and the corresponding density voxel grid, downsampled for visualization (slide 15).



Figure 9: Voxel grids are everywhere in medical imaging: a 3D brain volume with voxel (i, j, k) and a 2D MRI slice with pixel (i, j). Figure credits: www.hindawi.com/journals/cmmm/2015/450341.

Limits of the grid *(slides 17)*

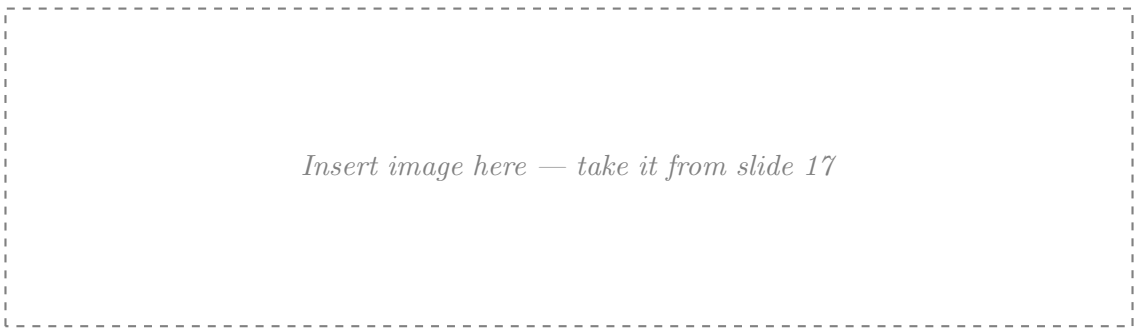


Figure 10: Slide 17: a real captured scene (red car) — grids "cannot model view-dependent effects" such as the car's reflections, and their size grows cubically with the scene dimension. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

Proposition 3.1 (Cubic memory growth)**.** *A dense grid with resolution N per axis and b bytes per voxel needs $N^3 b$ bytes. Doubling the resolution multiplies memory (and training/rendering cost per volume) by 8.*

Intuition

With σ and an RGB colour in 32-bit floats ($b = 16$ bytes), $N = 128$ costs 33.6 MB, $N = 512$ already 2.1 GB, and $N = 1024$ about 17.2 GB — for a scene that is mostly empty space. Together with the lack of view-dependent colour this is what motivates the MLP of the next section, and later sparse (Plenoxels) and hash-based (Instant-NGP) hybrids.

4 Neural Radiance Fields *(slides 18–20)*

Volumes: Rendering and Representation, Option 2: An MLP (slide 18). Instead of storing values on a lattice, store them in the weights of a neural network.

Definition 4.1 (Neural Radiance Field)**.** A NeRF is a multilayer perceptron (MLP)

$$f_\theta : \ (\boldsymbol{x}, \boldsymbol{\omega}) \longmapsto (\sigma, \boldsymbol{c}), \qquad \sigma \in \mathbb{R}^+, \quad \boldsymbol{c} \in [0,1]^3,$$

such that (1) given an input point it predicts the density, and (2) given a point *and* a viewing direction it predicts the radiance (colour). The scene is represented *entirely* by the weights θ.

Proposition 4.1 (Density is view-independent by design)**.** *If $\boldsymbol{\omega}$ is injected only after the layer that produces σ, then $\sigma = \sigma(\boldsymbol{x})$ does not depend on $\boldsymbol{\omega}$, while $\boldsymbol{c} = \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega})$ may.*

Proof. σ is computed by a function composition that has $\boldsymbol{x}$ (or features of it) as its only input; $\boldsymbol{\omega}$ is concatenated afterwards, so no path from $\boldsymbol{\omega}$ to σ exists in the computation graph. □

Intuition

Physically, whether there is matter at a point cannot depend on where you look from, whereas the colour of a shiny surface can. Slide 18 asks us to ensure this "with designing the MLP that density only depends on $\boldsymbol{x}$"; this is what the architecture of Figure 11 does. This is a built-in inductive bias which makes the 3D structure consistent across views.

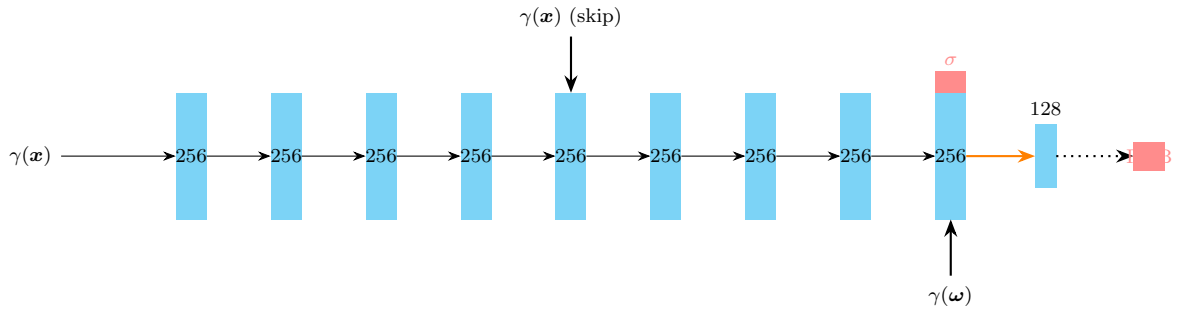


Figure 11: The NeRF MLP (redrawn from slides 18 and 24): fully connected blocks of width 256; the encoded position is fed again at the fifth block (skip connection); the density σ is read out *before* the viewing direction is concatenated; one extra 128-unit layer sees $\gamma(\boldsymbol{\omega})$ and outputs the RGB colour. Typical choices: ReLU hidden units, a non-negative activation (ReLU/softplus) for σ, sigmoid for RGB, so that $\sigma \in \mathbb{R}^+$ and $\boldsymbol{c} \in [0,1]^3$.

How to train this MLP? (slide 20). Key idea: *learn a 3D representation consistent with the observed views.* The next section turns this sentence into a loss.

5 Learning Neural Radiance Fields *(slides 21–24)*

5.1 From pixels to rays *(slides 21–22)*

Definition 5.1 (Camera ray)**.** Given a pixel $\boldsymbol{p} = (u, v)$ and camera parameters $\pi = (K, R, \boldsymbol{t}_c)$ (intrinsics K, world-from-camera rotation R and camera centre $\boldsymbol{t}_c$), the ray through $\boldsymbol{p}$ is

$$\boldsymbol{x}_t = \boldsymbol{o} + t\,\boldsymbol{d}, \qquad \boldsymbol{o} = \boldsymbol{t}_c, \quad \boldsymbol{d} = \frac{RK^{-1}(u, v, 1)^\top}{\|RK^{-1}(u, v, 1)^\top\|}, \quad t \geq 0.$$

The slides abbreviate this as the map (pixel, camera) $\to$ ray, and the viewing direction fed to the network is $\boldsymbol{\omega} = -\boldsymbol{d}$ (or $\boldsymbol{d}$, by convention).

Definition 5.2 (View-synthesis loss)**.** Let $\mathrm{render}(\boldsymbol{p}, \pi; f_\theta)$ be the volume-rendered colour (Theorem 2.1) of the ray through pixel $\boldsymbol{p}$ of the camera π when the volume is given by f_θ. The NeRF is trained by

$$\min_\theta \sum_I \sum_{\boldsymbol{p}} \left\| \mathrm{render}(\boldsymbol{p}, \pi; f_\theta) - I[\boldsymbol{p}] \right\|^2,$$

where I runs over the training images and $\boldsymbol{p}$ over their pixels.

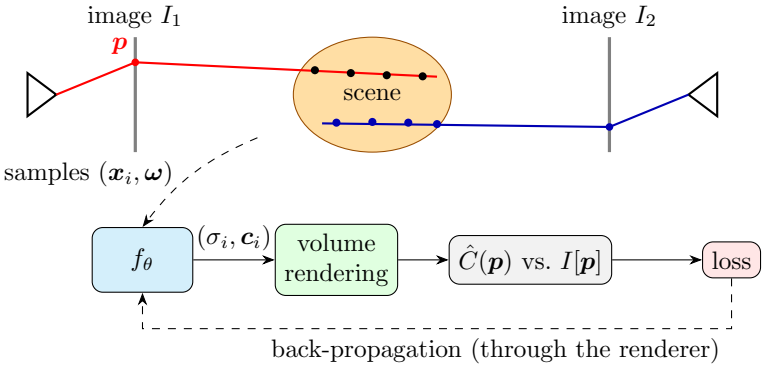


Figure 12: Learning a NeRF (redrawn and simplified from slides 21–22). Rays from two cameras are marched; the 3D points are sent to f_θ, whose outputs are composited by volume rendering and compared with the observed pixel. The gradient flows back through the (differentiable) renderer to θ.

Intuition

The loss never mentions 3D. It only says: "if I shoot the ray of pixel $\boldsymbol{p}$ through my current cloud, I should get the pixel's colour." But a 3D point on the visible surface lies on the rays of many cameras, and the only way to be right for *all* of them simultaneously is to put density at the correct place with a colour that is consistent across views. This is why slide 23 requires typically more than 50 images with *known* camera parameters (in practice obtained with structure-from-motion).

5.2 Mini-batches of rays *(slides 23)*

A single image already has $H \times W$ rays (e.g. $800 \times 800 = 640{,}000$) and each ray needs many network queries (Section 7), so we cannot back-propagate through all of them. "In practice, randomly sample a few pixels (rays) per optimisation iteration."

Proposition 5.1 (Unbiased stochastic gradients)**.** *Let* $\mathcal{L}(\theta) = \frac{1}{|\mathcal{P}|}\sum_{(I,\boldsymbol{p})\in\mathcal{P}} \ell_{I,\boldsymbol{p}}(\theta)$ *be the average per-ray loss over all training pixels* $\mathcal{P}$*, and* B *a batch of* m *pixels drawn uniformly (with replacement). Then* $\mathbb{E}_B\big[\frac{1}{m}\sum_{b\in B}\nabla\ell_b\big] = \nabla\mathcal{L}(\theta)$.

Proof. Each draw is uniform, so $\mathbb{E}[\nabla\ell_b] = \frac{1}{|\mathcal{P}|}\sum_{p\in\mathcal{P}}\nabla\ell_p = \nabla\mathcal{L}$; the average of m such expectations is the same. □

Intuition

It is the usual stochastic gradient descent, except that the "training examples" are individual rays, not whole images. Random rays from random images also decorrelate the updates, much like random mini-batches.

Algorithm 5.1 (NeRF training loop, slide 24)**.**

1. **Acquire** multiple images of a scene with associated camera viewpoints.
2. **Design** a neural network $(\sigma, \boldsymbol{c}) = f_\theta(\boldsymbol{x}, \boldsymbol{\omega})$.
3. **Train** with a view-synthesis loss using volume rendering: repeat *(a)* draw a batch of (image, pixel) pairs; *(b)* build their rays; *(c)* sample points along each ray; *(d)* query f_θ; *(e)* composite with Theorem 2.1; *(f)* compute the squared error to the pixel colours; *(g)* update θ by gradient descent.

Insert image here — take it from slide 24

Figure 13: Step 1 of the recipe: many camera viewpoints around the object (here a drum set); each camera is paired with its image. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

6 Learning Neural Radiance Fields: An Initial Attempt *(slides 25)*

Feeding the raw coordinates $(\boldsymbol{x}, \boldsymbol{\omega})$ to the MLP and training as above gives disappointing results (slide 25): from 100 training images the network produces *blurry* novel views — **it cannot capture high-frequency details**.



Figure 14: Slide 25: 100 training images → optimised neural net → novel-view rendering of the lego scene (blurry result; first attempt without positional encoding). Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

7 Modeling High Frequencies: A Toy Problem *(slides 26–30)*

The slides isolate the phenomenon on a 2D toy problem: regress an image, $(x, y) \to f_\theta \to (r, g, b)$ (slide 26). A "standard" MLP returns a smooth, blurry image, because ReLU MLPs trained by gradient descent fit low frequencies first (*spectral bias*). The remedy: lift the input to a vector of sines and cosines at geometrically increasing frequencies.

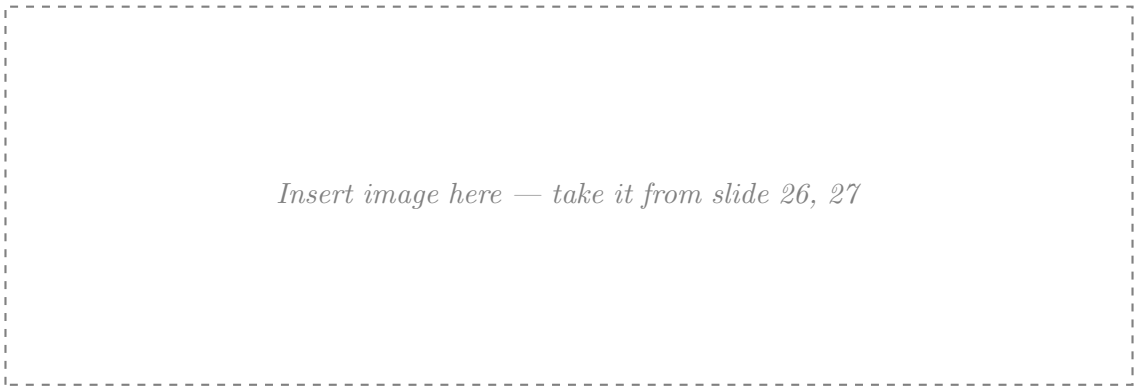


Figure 15: Image regression toy problem: input image, result with a "standard" MLP (blurry), and result using positional encoding (sharp). Figure credits: Tancik et al., NeurIPS 2020 (Fourier Features).

Definition 7.1 (Positional encoding)**.** For a scalar or vector $\boldsymbol{v}$ (applied coordinate-wise) and a number of octaves L,

$$\gamma(\boldsymbol{v}) = \big(\boldsymbol{v},\ \sin(\boldsymbol{v}), \cos(\boldsymbol{v}),\ \sin(2\boldsymbol{v}), \cos(2\boldsymbol{v}),\ \sin(4\boldsymbol{v}), \cos(4\boldsymbol{v}), \ldots,\ \sin(2^{L-1}\boldsymbol{v}), \cos(2^{L-1}\boldsymbol{v})\big).$$

(The slide lists the sin / cos terms; the original NeRF also multiplies the argument by π, i.e. $\sin(2^k \pi v)$, for coordinates normalised to $[-1, 1]$, and uses $L = 10$ for positions and $L = 4$ for directions. Whether $\boldsymbol{v}$ itself is kept is an implementation detail.) The network is now trained on $\gamma(\boldsymbol{x})$ and $\gamma(\boldsymbol{\omega})$ instead of $\boldsymbol{x}$ and $\boldsymbol{\omega}$.

Lemma 7.1 (γ induces a shift-invariant similarity)**.** *For scalars v, v',*

$$\langle \gamma(v), \gamma(v') \rangle_{\sin/\cos} = \sum_{k=0}^{L-1} \cos\big(2^k (v - v')\big)$$

which depends only on $v - v'$. For vectors, sum over the coordinates.

Proof. $\sin a \sin b + \cos a \cos b = \cos(a - b)$ applied with $a = 2^k v$, $b = 2^k v'$ for each k. □

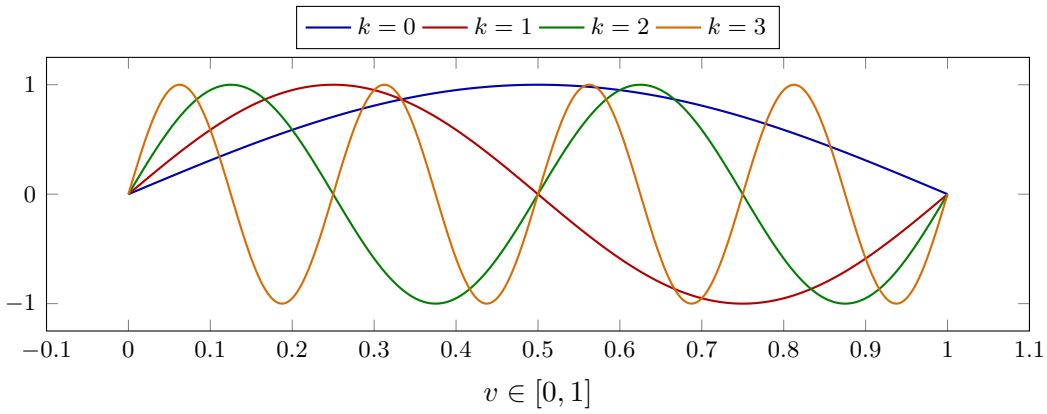


Figure 16: Original illustration: the first four components $\sin(2^k \pi v)$ of the positional encoding. Each octave doubles the frequency, so a few components span a wide range of scales; with $L = 10$ the finest component has a period of $2/2^9 = 1/256$ of the normalised coordinate range.

Intuition

Why does this help? Without encoding, two nearby points have nearly identical inputs, so the network treats them nearly identically (it is "smooth" in $\boldsymbol{x}$). After encoding, the high-frequency components $\sin(2^k \cdot)$ with large k change very fast with $\boldsymbol{x}$: two points 0.004 apart look completely different to the network, which can then give them different densities and colours. Lemma **??** says the network's notion of "similar points" becomes a band-limited kernel with bandwidth $\approx 2^{L-1}$ that we can tune, rather than a very smooth one. This is the Fourier-features argument of Tancik et al.: "Fourier features let networks learn high-frequency functions in low-dimensional domains".

Insert image here — take it from slide 28

Figure 17: Slide 28: with positional encodings at the input, the same pipeline (100 training images → optimised neural net → novel views) produces a sharp lego bulldozer — "a great example that execution matters". Figure credits: Mildenhall et al., ECCV 2020 (NeRF).



Figure 18: Slide 29: NeRF (naive) vs. NeRF (with positional encoding) on the "fern" scene — the encoded version recovers leaf detail. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).



Figure 19: Slide 30 (video frame): standard input vs. Fourier-feature input for a 3D shape, a 3D MRI and a 3D NeRF. Video credits: Tancik and Srinivasan et al., NeurIPS 2020 (Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains).

8 Uniform to Hierarchical Sampling *(slides 31–33)*

The rendering sum of Theorem 2.1 is a quadrature of the volume-rendering integral along the ray, and each term costs one MLP evaluation. Uniform spacing (slide 31) is wasteful: many samples fall in empty space or behind an opaque surface, where the weight w_i is tiny. Slides 32–33 therefore reuse

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_i w_i \, \boldsymbol{c}_i, \qquad w_i = T_i\big(1 - e^{-\sigma_{t_i} \Delta t}\big),$$

where the *weights indicate how much each segment contributes to the pixel value* (red = high, yellow = low). Strategy: (1) sample points uniformly; (2) sample *more* points in the segments with large weights.

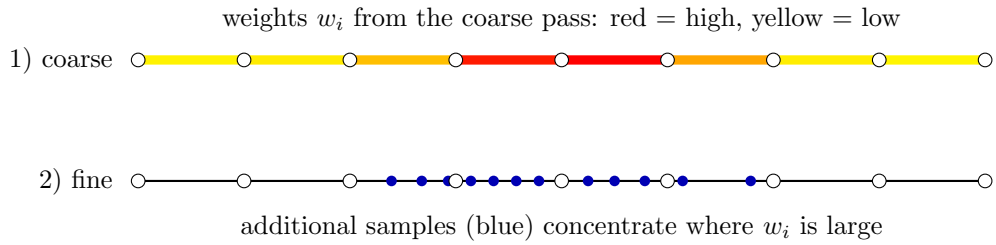


Figure 20: Hierarchical sampling (redrawn and simplified from slides 31–33). A first, uniform "coarse" pass computes the weights; a second pass draws extra samples with density proportional to them. (Sample positions are schematic.)

Definition 8.1 (Piecewise-constant PDF along the ray)**.** Normalise the coarse weights, $\hat{w}_i = w_i / \sum_j w_j$, and define the density on $[t_1, t_{N+1}]$ that equals $\hat{w}_i / \Delta t_i$ on the i-th segment and the cumulative distribution $F(t) = \int_{t_1}^{t} p(s)\,\mathrm{d}s$, which is piecewise linear.

Lemma 8.1 (Inverse-transform sampling)**.** *If $U \sim \mathcal{U}[0,1]$ and F is a strictly increasing CDF, then $t = F^{-1}(U)$ has CDF F.*

Proof. $\Pr[F^{-1}(U) \le t] = \Pr[U \le F(t)] = F(t)$, since F is monotonic and U is uniform. (For a piecewise-linear CDF with flat parts, take the left-continuous generalised inverse.) □

Algorithm 8.1 (Hierarchical sampling)**.**

1. Sample N_c points uniformly along the ray (one random point inside each of N_c equal bins in the original NeRF) and evaluate the network ("coarse" pass).
2. Compute the weights w_i and the normalised piecewise-constant PDF $\hat{w}_i$.
3. Draw N_f further points $t = F^{-1}(u_j)$ with $u_j \sim \mathcal{U}[0,1]$ (Lemma 8.1).
4. Evaluate the network on the sorted union of all $N_c + N_f$ points and render the pixel ("fine" pass). Original NeRF: $N_c = 64$, $N_f = 128$, with a coarse and a fine network, both supervised.

Example 8.1 (Inverse CDF by hand)**.** Four bins of unit length on $t \in [0, 4]$ with normalised weights $(0.1, 0.5, 0.3, 0.1)$ give the CDF values $(0, 0.1, 0.6, 0.9, 1)$ at $t = 0, 1, 2, 3, 4$. For $u = 0.5$: $F(t) = 0.5$ lies in bin 2 (between 0.1 and 0.6) at fraction $(0.5 - 0.1)/0.5 = 0.8$ of the way, so $t = 1 + 0.8 = 1.8$. Half of the random numbers $u \in [0.1, 0.6]$ fall in bin 2, so bin 2 receives half of the new samples although it is only a quarter of the ray length.

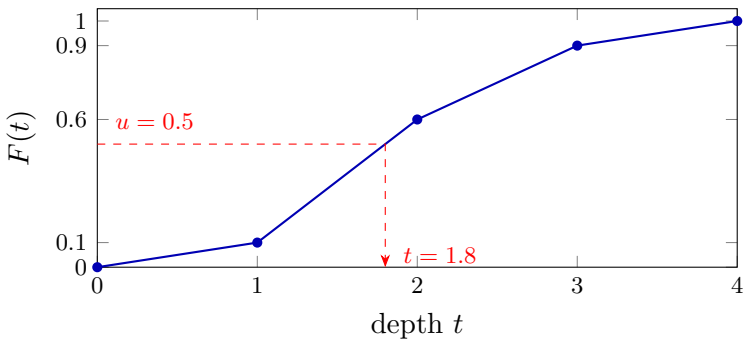


Figure 21: Original illustration: inverse-CDF sampling for the example above. The steep part of F (bin 2) is hit by many uniform u's.

Intuition

It is *importance sampling* with the weights as importance function: the weights tell us where the pixel colour is actually decided (typically near the first visible surface), and we spend our (expensive) network queries there. Empty space and occluded space, where $w_i \approx 0$, are nearly skipped. For a fixed budget this gives a sharper picture than uniformly spending all queries.

9 Depth vs Inverse-Depth Sampling *(slides 34–36)*

Uniform depth sampling (slide 34) is informative for scenes with limited depth variation, e.g. a camera orbiting an object. In a forward-facing, real scene the content can lie anywhere between a near point and infinity (slide 35: a plant very close, a wall far away); uniform spacing then places almost all samples far away, while near geometry gets very few. Alternative: sample *uniformly in* $1/d$ — a user-driven choice in NeRF: sampling in depth or in inverse-depth space.

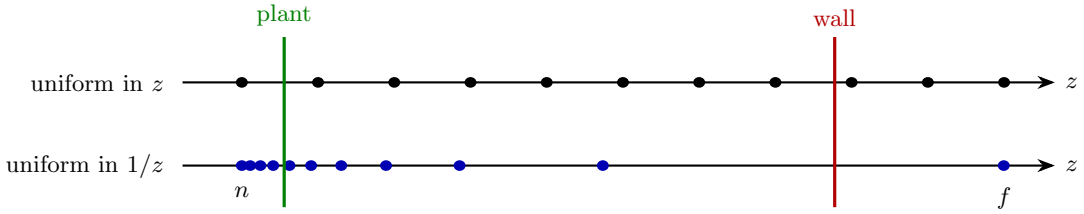


Figure 22: Original illustration, $n = 1$, $f = 10$, $N = 10$ intervals. Uniform depth puts 8 of the 11 points beyond $z = 3$; uniform inverse depth puts 7 of them within $z \leq 2.2$ and still reaches the far plane. The nearby plant is resolved by several samples in the second row.

9.1 Normalised device coordinates *(slides 36)*

The NeRF paper (Appendix C) maps the depth range into $[-1, 1]$ using normalised device coordinates (NDC) because real captures contain content "anywhere between the closest point and infinity". With near and far planes n, f and camera-space depth $z_c > 0$:

Proposition 9.1 (NDC depth is affine in inverse depth)**.**

$$z_{\text{nd}} = \frac{f+n}{f-n} - \frac{2fn}{(f-n)\,z_c} \in [-1, 1].$$

Hence sampling z_{nd} uniformly is the same as sampling $1/z_c$ uniformly. For $f \to \infty$, uniform inverse-depth samples are $z_i = \dfrac{n}{1 - i/N}$, whereas uniform depth samples are $z_i = n + \frac{i}{N}(f-n)$.

Proof. A perspective projection yields a depth of the form $z_{\text{nd}} = A + B/z_c$. The conditions $z_{\text{nd}}(n) = -1$ and $z_{\text{nd}}(f) = +1$ give $A + B/n = -1$ and $A + B/f = 1$, so $B = \frac{2}{1/f - 1/n} = -\frac{2fn}{f-n}$ and $A = 1 - B/f = \frac{f+n}{f-n}$. Since z_{nd} is affine in $1/z_c$, equal steps in z_{nd} are equal steps in $1/z_c$: $1/z_i = (1 - s_i)/n + s_i/f$ with $s_i = i/N$; letting $f \to \infty$ gives $z_i = n/(1 - s_i)$. □

Example 9.1. For $n = 1$, $f = 10$: uniform depth gives $z = 1, 1.9, 2.8, \ldots, 10$; uniform inverse depth gives $z = 1/(1 - 0.09i) = 1,\ 1.10,\ 1.22,\ 1.37,\ 1.56,\ 1.82,\ 2.17,\ 2.70,\ 3.57,\ 5.26,\ 10$. Spacing grows steadily: the first gap is 0.10, the last one 4.74.

Intuition

A camera sees a nearby thing and a far thing at very different pixel sizes: close geometry deserves a fine depth resolution, distant geometry does not need it (it moves little on the image as the camera moves — parallax is proportional to $1/z$). Sampling uniformly in $1/z$ spends samples proportionally to how much depth matters on the image, and lets the far plane go to infinity at the cost of one extra sample. A good detailed explanation of NDC is linked on slide 36 (`songho.ca/opengl/gl_projectionmatrix.html`).

10 Novel-View Synthesis and View-Dependent Effects *(slides 37–41)*

10.1 Novel-View Synthesis *(slides 37)*

Insert image here — take it from slide 37

Figure 23: Novel-view synthesis results of NeRF on real captured scenes (a triceratops skull and a mosaic water fountain): smooth camera paths through views never seen in training. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

10.2 View-Dependent Effects *(slides 38–40)*

Definition 10.1 (View-dependent effect)**.** A *view-dependent* effect is an appearance change that results from the viewing direction alone, while geometry and light stay fixed — specular highlights, reflections, glossy materials. In radiance-field terms: $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_1) \neq \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_2)$ for the *same* 3D point.

Proposition 10.1. *A representation with $L_e(\boldsymbol{x}, \boldsymbol{\omega}) \equiv \boldsymbol{c}(\boldsymbol{x})$ (the grid of Section 3, slide 13) cannot reproduce a scene in which the same 3D point has different colours in two training images.*

Proof. By construction $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_1) = \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_2)$; one value cannot match two different observations. The best it can do is average them, or distort the geometry (put density at wrong places) to explain the inconsistency. □

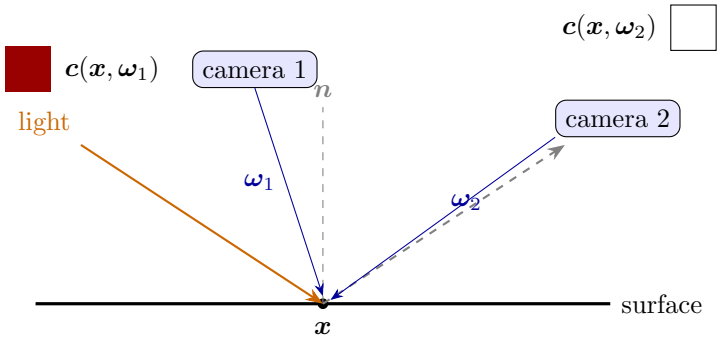


Figure 24: Original illustration: a glossy surface point seen from two directions. Camera 2 lies near the mirror direction of the light and sees a bright highlight; camera 1 sees only the dark body colour. A NeRF captures this through the dependence of $\boldsymbol{c}$ on $\boldsymbol{\omega}$.

Insert image here — take it from slide 38

Figure 25: Slide 38: same 3D point, different colour based on the viewing direction — zoomed patches from two views of the synthetic ship scene (left and right) and the colour as a function of the viewing direction. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

Insert image here — take it from slide 39, 40

Figure 26: Slides 39 and 40 (video frames): real scenes (a kitchen stove and a conference room) with highlights on metal and on the screen that move as the camera moves. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

Intuition

For a perfectly matte (Lambertian) surface, colour does not depend on the viewpoint and a grid with $\boldsymbol{c}(\boldsymbol{x})$ would be enough. Real materials are not matte: the highlight on a pan slides across it when you walk around. NeRF handles it because the network gets $\boldsymbol{\omega}$ as input, but only in the *last* layers (Proposition 4.1): "geometry first, appearance later".

10.3 Learning Neural Radiance Fields: a simple recipe *(slides 41)*

Slide 41 closes the NeRF part: positional encoding at the input, a plain MLP, volume rendering and a photometric loss. It is a "very simple" representation and learning approach which **inspired many, many extensions and follow-ups**; the rest of the lecture visits some of them.

11 Is ray sampling accurate? *(slides 42)*

So far one pixel = one ray through its centre. But a real pixel integrates light over a small *footprint* on the sensor.

Definition 11.1 (Pixel value with a box filter)**.** The value of pixel $\boldsymbol{p}$ is $I[\boldsymbol{p}] = \iint_{\text{pixel}} \text{render}(\boldsymbol{q})\, \mathrm{d}\boldsymbol{q}$ (averaged over the pixel area), whereas the *point-sampled* approximation uses only $\text{render}(\boldsymbol{p}_{\text{center}})$.

Insert image here — take it from slide 42

Figure 27: Slide 42: a checkerboard plane rendered with one ray through each pixel centre (aliased, moiré pattern near the horizon) versus averaged over the whole pixel (box filter, smooth).

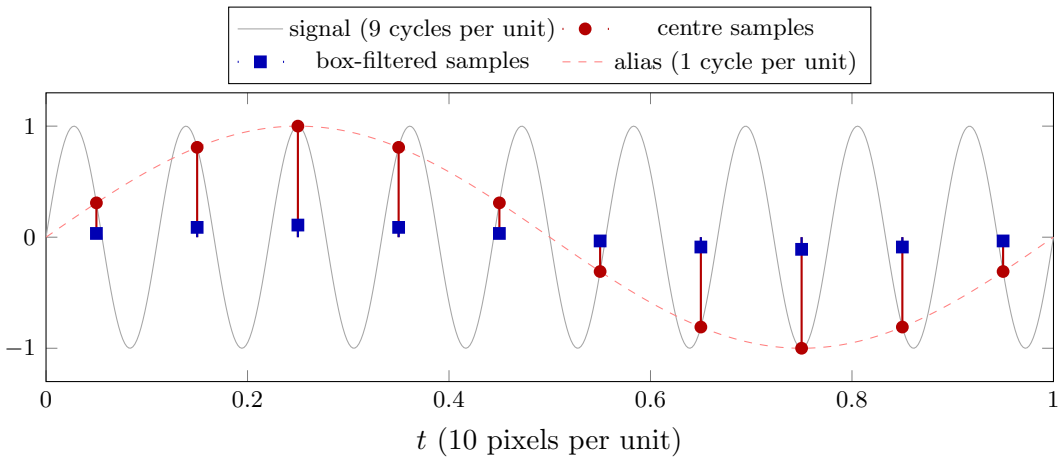


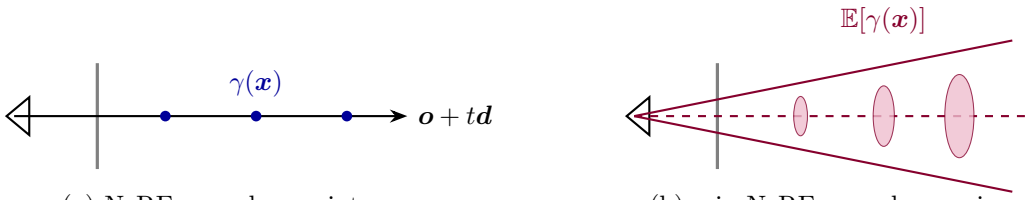
Figure 28: Original illustration of aliasing. The signal $\sin(2\pi \cdot 9t)$ is far above the Nyquist limit of 5 cycles/unit for 10 pixels. Centre sampling returns a full-amplitude *false* low-frequency pattern $\sin(2\pi t)$ (the moiré). Averaging over the pixel width 0.1 multiplies the amplitude by $\text{sinc} = \sin(0.9\pi)/(0.9\pi) \approx 0.11$, so the unresolvable detail is attenuated instead of impersonating a coarse one.

Intuition

A single ray through the pixel centre answers "what colour is the world at this exact point?" whereas the camera asked "what is the *average* colour of this little patch?". When the patch covers many fine details (distant checkerboard squares, or a scene viewed from far away) the two answers differ a lot. NeRF trained on images of different resolutions or distances then gets conflicting supervision from the same 3D point: sharp renders at close range look jagged/aliased, and blurry when it tries to compromise.

12 Pixels as Cones of Rays *(slides 43–46)*

Mip-NeRF (Barron et al., ICCV 2021) treats a pixel as a *cone* (a frustum of cone between two depths t_0, t_1) instead of a line. Slide 43's key sentence: *instead of the position encoding of a single sample, use the expected position encoding.*



(a) NeRF: encode a point (b) mip-NeRF: encode a region

Figure 29: Pixels as cones of rays (redrawn from slide 43). A conical frustum covering the pixel footprint between two depths is summarised by a Gaussian with mean $\boldsymbol{\mu}$ and covariance Σ; the network receives the expected (integrated) positional encoding of that Gaussian. Figure credits: Barron et al., ICCV 2021 (mip-NeRF).

Lemma 12.1 (Integrated positional encoding)**.** *Let $X \sim \mathcal{N}(\mu, s^2)$ be scalar. For every frequency $a > 0$,*

$$\mathbb{E}[\sin(aX)] = \sin(a\mu)\, e^{-a^2 s^2/2}, \qquad \mathbb{E}[\cos(aX)] = \cos(a\mu)\, e^{-a^2 s^2/2}.$$

Hence for $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ coordinate-wise with $\Sigma_{jj} = s_j^2$ the expected encoding of the k-th band is $\sin(2^k \mu_j)\exp(-\frac{1}{2}4^k s_j^2)$ and $\cos(2^k \mu_j)\exp(-\frac{1}{2}4^k s_j^2)$.

Proof. $\mathbb{E}[e^{iaX}] = e^{ia\mu - a^2 s^2/2}$ is the characteristic function of a Gaussian. Taking imaginary and real parts gives $\mathbb{E}[\sin aX] = \Im$ and $\mathbb{E}[\cos aX] = \Re$ of $e^{ia\mu}e^{-a^2 s^2/2}$. With $a = 2^k$ we have $a^2 = 4^k$. □

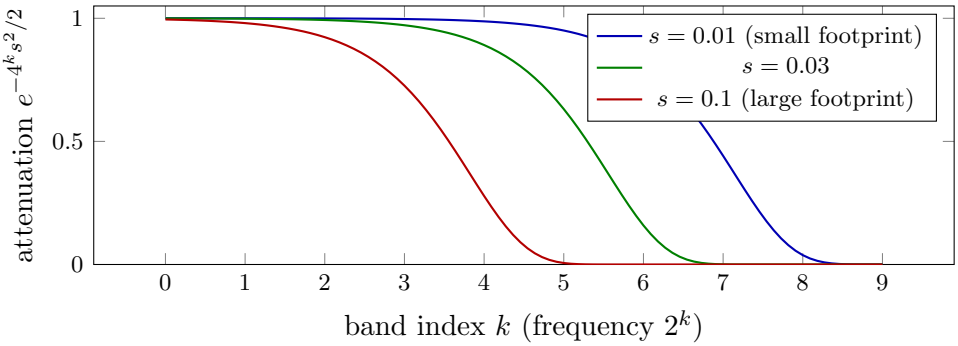


Figure 30: Original illustration of Lemma 12.1: the factor multiplying band k of the integrated encoding. A larger footprint s switches off the high-frequency bands, while a small footprint leaves them intact.

Intuition

When the pixel's footprint is large, details finer than the footprint are averaged away in the real photograph. The integrated encoding mimics this: bands whose wavelength is shorter than the footprint are smoothly *damped to zero*, while coarse bands pass through untouched; the network never sees a high frequency it could not have observed. It also now knows *how large* the region is, so one network can be trained on images at several resolutions. Slide 45 (mip-NeRF paper) shows that this improves the fine details at lower resolutions with the PSNR/SSIM-type numbers printed next to the crops.

Insert image here — take it from slide 45

Figure 31: Slide 45: ground truth, NeRF, NeRF + (area, centre, misc), and mip-NeRF crops of the drum and microphone scenes at several scales, with the error scores printed next to them. Figure credits: Barron et al., ICCV 2021 (mip-NeRF).



Figure 32: Slide 46 (video frames): two renderings of the microphone scene compared side by side while the scale changes. Video credits: Barron et al., ICCV 2021 (mip-NeRF).

13 Volumes: Rendering and Representation — Option 2: An MLP, revisited *(slides 47)*

Slide 47 puts the MLP in perspective: it is **a very flexible representation** (continuous, compact, able to model view-dependent effects) but **can be slow to train** — several hours for a single scene.

Proposition 13.1 (Cost of one training iteration)**.** *If a batch has m rays and each ray needs $N_c + N_f$ network evaluations (Section 7), one forward–backward pass evaluates the MLP $m(N_c + N_f)$ times.*

Intuition

With e.g. $m = 4096$ rays and $N_c + N_f = 64 + 128 = 192$ samples, one iteration requires about 786,000 evaluations of a 9-layer, 256-wide MLP — and hundreds of thousands of iterations are needed. Everything that follows tries to make the *query* cheaper: put the information in a grid, which only needs a few memory reads per point, instead of a deep network.

14 Option 3: Voxels with view-dependent radiance — Do We Need Neural Radiance? *(slides 48–51)*

Plenoxels (Fridovich-Keil, Yu et al., CVPR 2022) answer "*do we need neural radiance?*" with *no*: keep the sparse voxel grid of Option 1 but fix its two weaknesses — the missing view dependence and the cubic memory — without any neural network (slide 48).

14.1 Spherical harmonics for view-dependent colour *(slides 49)*

Definition 14.1 (Spherical harmonics)**.** The (real) spherical harmonics $Y_\ell^m : \mathbb{S}^2 \to \mathbb{R}$, $\ell \geq 0$, $-\ell \leq m \leq \ell$, are an orthonormal basis of square-integrable scalar functions on the sphere: $\int_{\mathbb{S}^2} Y_\ell^m Y_{\ell'}^{m'} \, \mathrm{d}\boldsymbol{\omega} = \delta_{\ell\ell'}\delta_{mm'}$. Degree ℓ contributes $2\ell + 1$ functions, so truncating at degree $\ell_{\max}$ keeps $(\ell_{\max} + 1)^2$ of them.

Definition 14.2 (View-dependent colour from basis coefficients)**.** Each voxel stores, for each

colour channel, coefficients k_ℓ^m and the colour seen from direction $\boldsymbol{\omega}$ is

$$\boldsymbol{c}_{\text{ch}}(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{\ell \le \ell_{\max}} \sum_{m=-\ell}^{\ell} k_{\ell,\text{ch}}^m(\boldsymbol{x})\, Y_\ell^m(\boldsymbol{\omega})$$

(followed in practice by a sigmoid or clamp to enforce $[0,1]$). Plenoxels uses $\ell_{\max} = 2$: 9 coefficients per channel, 27 for RGB, plus 1 for the density, i.e. 28 numbers per voxel.

Lemma 14.1 (Linearity in the stored coefficients)**.** *The colour is a linear function of the coefficients, so trilinear interpolation of the coefficients of the eight corners and evaluation at $\boldsymbol{\omega}$ commutes: interpolating coefficients first and then evaluating $\sum kY$ gives the same as evaluating each corner's colour first and then interpolating. Moreover $\partial \boldsymbol{c}_{\text{ch}} / \partial k_\ell^m = Y_\ell^m(\boldsymbol{\omega})$ (times the interpolation weight).*

Proof. Both operations are linear combinations of the corner coefficients with weights λ_{ijk} (Lemma 3.1) and $Y_\ell^m(\boldsymbol{\omega})$, which do not depend on one another. The derivative is read off the formula. □

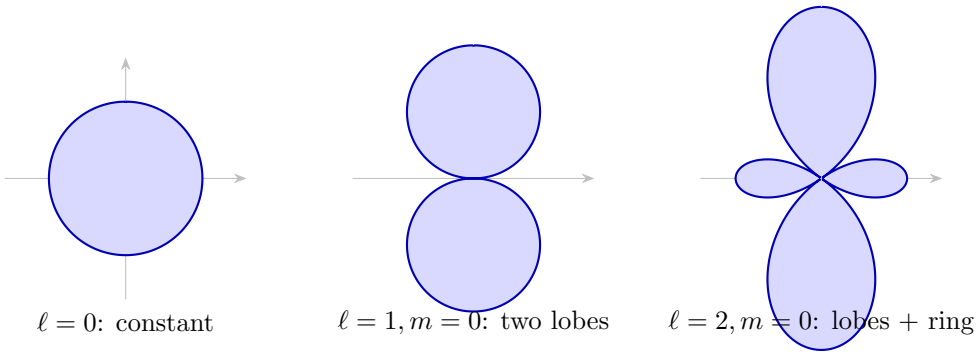


Figure 33: Original illustration (slide 49 shows the full pyramid of harmonics): polar plots of the magnitude $|Y_\ell^0(\theta)|$ of the zonal harmonics for $\ell = 0, 1, 2$ (vertical axis = polar axis; lobes alternate in sign). Higher degrees add angular detail, i.e. faster variation of colour with the viewing direction.

Intuition

Spherical harmonics are the Fourier series of the sphere. The $\ell = 0$ coefficient is the average, view-independent colour (the "diffuse" part); $\ell = 1$ lets the colour tilt towards one side of the sphere, and $\ell = 2$ adds richer variation like highlight shapes. Storing a handful of coefficients per voxel is therefore a compact table of *how the colour of this voxel changes around the sphere*, so the voxel no longer has the single colour of Option 1 — exactly the missing ingredient in slide 17 —, and no network is needed to get it.

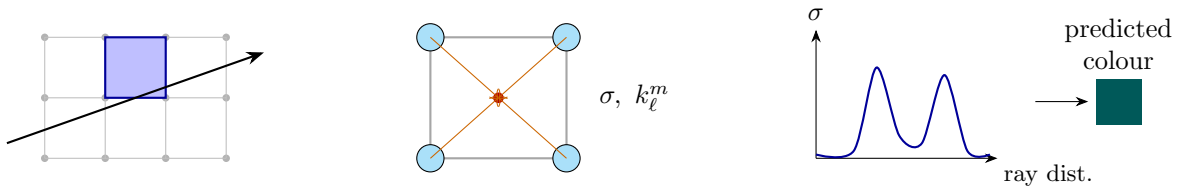
14.2 The Plenoxels pipeline *(slides 50)*

Algorithm 14.1 (Plenoxels, slide 50)**.**

1. **Sparse voxel grid.** At each voxel store the density and the spherical-harmonic coefficients for view-dependent colour.
2. **Trilinear interpolation** of density and coefficients at every sample along the ray (Lemma 3.1); evaluate $Y_\ell^m(\boldsymbol{\omega})$ to get the colour.
3. **Volumetric rendering** (Theorem 2.1) to obtain the predicted colour.

4. **Optimisation:** minimise $\mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{TV}$ over all voxel values directly by gradient descent.
5. For efficiency, **prune voxels with very low density** (they carry no visible information).

Here $\mathcal{L}_{\text{recon}}$ is the same photometric loss as for NeRF and $\mathcal{L}_{TV}$ a total-variation regulariser, which penalises large differences between neighbouring voxels, e.g. $\mathcal{L}_{TV} \propto \sum_v \sum_{d \in \{x,y,z\}} |f(v) - f(v+e_d)|$ for each stored quantity f.



(a) sparse voxel grid (b) trilinear interpolation (c) volumetric rendering

Figure 34: The Plenoxels pipeline (redrawn from slide 50; original figure by Fridovich-Keil, Yu et al., CVPR 2022): (a) sparse voxel grid; (b) trilinear interpolation of the eight corner values; (c) volumetric rendering along the ray; (d, not drawn) optimisation of the voxel values with $\mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{TV}$.

Intuition

There is nothing to "forward" through a network: a query is eight memory reads and a few multiplications, and the gradient w.r.t. a voxel value is just its interpolation weight (Lemma 3.1(iv)). That is why training takes minutes rather than hours. The price is more memory (28 numbers per voxel) and some noise when views are few, which TV smoothing fights.

14.3 NeRF vs. Plenoxels *(slides 51)*

Insert image here — take it from slide 51

Figure 35: Slide 51 (video frames): training NeRF (left) and Plenoxels (right) side by side with a `minutes:seconds` timer; Plenoxels reaches a recognisable result long before NeRF. Figure credits: Fridovich-Keil, Yu et al., CVPR 2022 (Plenoxels).

15 Towards Hybrid Representations *(slides 52–54)*

Option 4: feature hash-grid + lightweight MLP (Instant-NGP, Müller et al., SIGGRAPH 2022). The idea combines the two worlds: store *learned features* in a (hashed) grid for fast lookups, and let a *tiny* MLP map those features and the direction to density and colour (slide 53).

Definition 15.1 (Multiresolution hash encoding)**.** Use L resolution levels, with grid resolutions $N_\ell = \lfloor N_{\min} b^\ell \rfloor$ (geometric growth factor b). Level ℓ owns a table $\theta_\ell \in \mathbb{R}^{T \times F}$ of T entries with F features each. For $\boldsymbol{x} \in [0,1]^d$:

1. *Hashing of voxel vertices:* the 2^d integer vertices $\boldsymbol{c}$ of the cell containing $\boldsymbol{x}N_\ell$ are mapped to table slots by $h(\boldsymbol{c}) = \left(\bigoplus_{i=1}^{d} c_i \pi_i\right) \bmod T$, with large primes π_i ($\pi_1 = 1$, $\pi_2 = 2654435761$, $\pi_3 = 805459861$ in the paper) and $\oplus$ the bitwise XOR. If the level has at most T vertices it is indexed directly.
2. *Lookup* of the F-dimensional features of the vertices.
3. *(Multi)linear interpolation* of those features with the weights of Lemma 3.1 (the 2D version for $d = 2$ in the slide).
4. *Concatenation* of the L interpolated vectors (giving $L \cdot F$ numbers) with auxiliary inputs ξ (e.g. the encoded view direction): $\boldsymbol{y} \in \mathbb{R}^{LF+E}$.
5. *Neural network:* a tiny MLP $m(\boldsymbol{y}; \Phi)$ outputs density and colour.

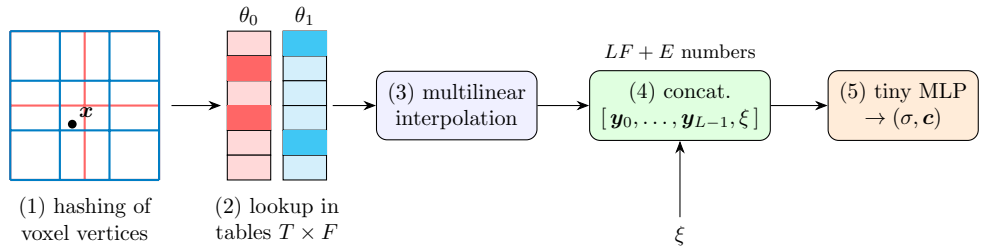


Figure 36: The hash-grid encoding (redrawn from slides 52–53; original figure by Müller et al., SIGGRAPH 2022). Two levels of a 2D toy grid are shown. Each level has its own small table; vertices from many places of space can share a table slot (hash collision).

Proposition 15.1 (Memory is fixed by the hyper-parameters, not by the scene)**.** *The encoding has $L \cdot T \cdot F$ trainable numbers regardless of the resolution N_L reached by the finest level, whereas a dense grid of the same finest resolution needs $(N_L + 1)^d$ vertices.*

Intuition

Example with the typical NeRF-style values $L = 16$, $F = 2$, $T = 2^{19}$ (as reported for Instant-NGP): $16 \cdot 524{,}288 \cdot 2 \approx 16.8$M numbers — about 34 MB in half precision — whether the finest resolution is 512 or 2048 per axis; a dense 2048^3 grid with 4 values per voxel in single precision would need ≈ 137 GB. How do we get away with this? Real scenes are mostly empty: only $\sim N^2$ vertices near surfaces matter, which is of the order of the table size, and when two vertices collide the network and the gradients (dominated by the vertex that matters more for the image) sort it out; the coarse levels have no collisions and give context. The tiny MLP (e.g. 1–2 hidden layers of 64 units in the paper) is cheap because the features already carry the spatial information.

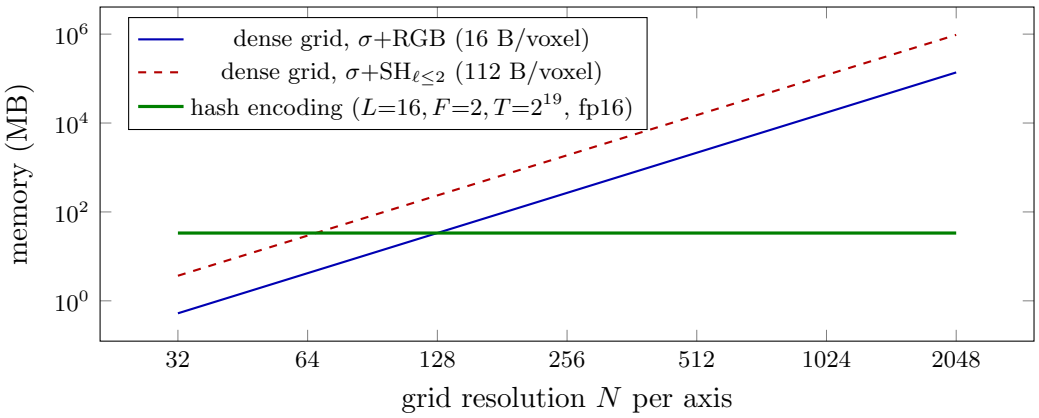


Figure 37: Original illustration: memory of dense grids grows as N^3 (a straight line of slope 3 on this log-log plot) while the hash encoding stays constant (assumed values $L = 16$, $F = 2$, $T = 2^{19}$, half precision). Plenoxels avoids most of the dense cost by storing only occupied voxels (sparse grid), so its real footprint is much smaller than the dashed line.

Insert image here — take it from slide 54

Figure 38: Slide 54 (video frames): Instant-NGP training on eight different scenes/images with an "Elapsed training time" counter — a usable result appears after seconds. Video credits: Müller et al., SIGGRAPH 2022 (Instant-NGP).

Remark. Slide 53 notes that the core idea — a hash function that efficiently stores per-voxel features, plus a tiny MLP that maps a query point's features (and direction) to colour and density — is applicable to the efficient modelling of general 2D/3D signals, not only radiance fields (images, signed distance functions, ...).

16 Summary *(slides 55)*

Slide 55: *last lecture*: volume rendering; *today*: neural representations for volumes, and training via view synthesis. The common recipe is: (representation of σ, L_e) $\rightarrow$ differentiable ray marching $\rightarrow$ photometric loss with posed images.



Figure 39: Slide 55: training images → MLP / sparse voxel grid with spherical harmonics → novel view of the lego scene. Figure credits: Mildenhall et al. (NeRF), Fridovich-Keil, Yu et al. (Plenoxels).

Method	**Representation**	**View-dep. colour**	**Key idea**	**Training time (slides)**	**Main limitation**
Voxel grid (Opt. 1)	$\sigma[x, y, z]$, $\boldsymbol{c}[x, y, z]$, trilinear interp.	No ($L_e \equiv \boldsymbol{c}(\boldsymbol{x})$)	Store values on a lattice; differentiable lookups	fast queries	cubic memory; no view effects
NeRF (Opt. 2)	MLP $f_\theta(\boldsymbol{x}, \boldsymbol{\omega}) \to (\sigma, \boldsymbol{c})$	Yes, $\boldsymbol{\omega}$ injected late	Photometric loss through volume rendering; positional encoding; hierarchical sampling	several hours per scene	slow to train; blurry without γ
mip-NeRF	NeRF with cones and integrated encoding $\mathbb{E}[\gamma(\boldsymbol{x})]$	Yes	Pixels are footprints, not points; anti-aliasing across scales	like NeRF	still an MLP: slow
Plenoxels (Opt. 3)	sparse voxels: σ + SH coefficients	Yes (spherical harmonics)	No network; optimise voxels directly with $\mathcal{L}_{\text{recon}} + \lambda\mathcal{L}_{TV}$; prune empty voxels	minutes	large memory per voxel (28 numbers); noise with few views
Instant-NGP (Opt. 4)	multires hash-grid features + tiny MLP	Yes (direction goes into the MLP)	Fixed-size learned hash tables; tiny MLP	seconds to minutes	hash collisions; fixed table size

Table 1: Comparison of the representations seen in the lecture. All share the same rendering equation (Theorem 2.1) and photometric training signal; they differ in how $\sigma(\boldsymbol{x})$ and $L_e(\boldsymbol{x}, \boldsymbol{\omega})$ are stored and queried.

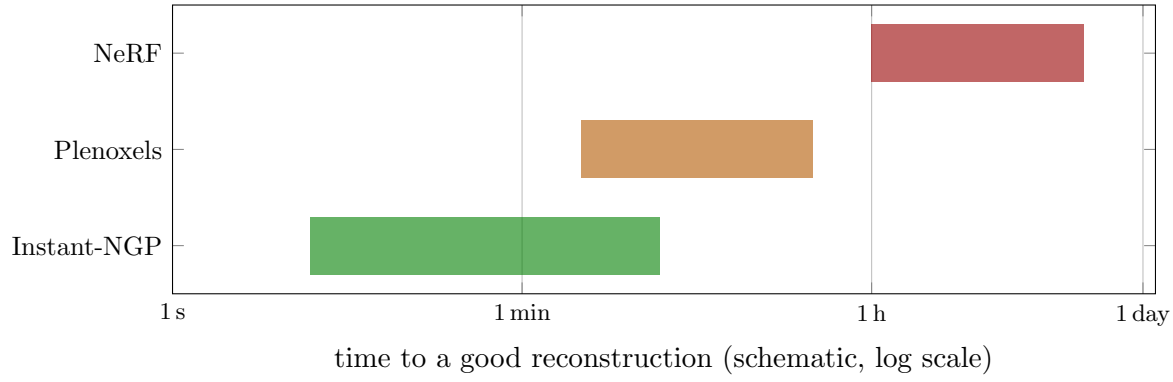


Figure 40: Qualitative comparison (schematic, orders of magnitude only; not measured here): the slides only state "several hours" for NeRF, and show timers of minutes for Plenoxels and seconds for Instant-NGP. Moving information from deep networks into explicit/hashed grids buys one to three orders of magnitude in training time.

Key take-aways

- **Volume rendering is the differentiable bridge.** Density and emitted light along a ray give a pixel via $L = \sum_i T_i(1 - e^{-\sigma_i \Delta t_i})\boldsymbol{c}_i$ (+ background $T_{N+1}L_{\text{bg}}$), with weights summing to one; gradients flow to densities, colours and, if these are differentiable functions of position and direction, to the camera.
- **A radiance field is any function $(\boldsymbol{x}, \boldsymbol{\omega}) \mapsto (\sigma, \boldsymbol{c})$.** It can be a grid (fast but cubic memory, no view effects), an MLP (compact, flexible, slow), voxels with spherical harmonics, or a hash-grid with a tiny MLP; training is always "match the posed images" (view synthesis loss on randomly sampled rays).
- **Details make or break NeRF:** positional encoding γ overcomes spectral bias; density depends on $\boldsymbol{x}$ only while colour also on $\boldsymbol{\omega}$; hierarchical (importance) sampling spends queries where w_i is large; inverse-depth/NDC sampling handles unbounded scenes.
- **Pixels are areas, not points:** mip-NeRF replaces the encoding of a single point by the expected (integrated) encoding of the pixel's conical frustum, which damps frequencies finer than the footprint and removes aliasing.
- **Hybrid representations trade neural computation for memory lookups:** sparse voxels + SH (Plenoxels) and multiresolution hash features + tiny MLP (Instant-NGP) reduce training from hours to minutes or seconds.

Credits. Slides adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani). Ray-marching figures by Ioannis (Yannis) Gkioulekas. Results and diagrams from Mildenhall et al., ECCV 2020 (NeRF); Tancik et al., NeurIPS 2020 (Fourier Features); Barron et al., ICCV 2021 (mip-NeRF); Fridovich-Keil, Yu et al., CVPR 2022 (Plenoxels); Müller et al., SIGGRAPH 2022 (Instant-NGP).